Project Title Cloud-Hosted Document Management System with Secure File Sharing, Version Control, and Access Management

Submitted in partial fulfillment of the requirements for the award of the degree of
Master of Computer Application (MCA) - Cloud Computing

By

Student Name: ___Sakshi Rana_____

Enrollment No: ___ 024MCA110041_____

Under the guidance of

Guide/Mentor Name: _____ Hari Krishna _____

Certificate

Copy to the certificate received from Qollabb to be pasted here.

Declaration

I, **_Sakshi Rana-**, hereby solemnly declare that the project report titled **"** Cloud-Hosted Document Management System with Secure File Sharing, Version Control, and Access Management **__"** submitted in partial fulfillment of the requirements for the award of the degree of **Master of Computer Application (MCA)** is my original work.

I further declare that:

- This project has been carried out by me during the academic year 2026_____ under the supervision of _____**Hari Krishan**_____ **(Guide/Mentor Name)**.
- The work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma, or certification.
- All sources of information used in this report have been duly acknowledged and referenced in accordance with academic ethics and plagiarism norms.
- The data presented in this report is authentic to the best of my knowledge and has not been fabricated or manipulated.
- I understand that if any part of this declaration is found to be false, my project may be rejected and disciplinary action may be taken as per institutional rules.

Place: _Pune_____
Date: ___07/07/2026_____

Student Signature: _____Sakshi Rana_____
Student Name: _____Sakshi Rana_____
Enrollment No: _____ O24MCA110041_____

Acknowledgement

Write a short paragraph thanking your guide(Mentor), faculty members and organization, for their support.

I express my sincere gratitude to my guide, faculty members, and all those who supported me during the development of this project. Their guidance, encouragement, and constructive suggestions helped me understand the academic and technical aspects of the work in a disciplined manner. I also acknowledge the role of cloud computing tools, software resources, and learning materials that contributed to the successful completion of this report

Abstract / Executive Summary (150–250 words)

This project report presents the design and implementation approach for a cloud-hosted document management system that supports secure file upload, centralized storage, controlled sharing, version control, and access management. The system addresses the practical challenges of fragmented storage, weak traceability, and limited security found in traditional file handling methods. It adopts a structured software engineering approach and applies cloud computing principles to build a scalable and implementation-focused solution. The proposed architecture includes a metadata layer, application logic layer, cloud object storage integration, encryption controls, permission management, monitoring support, and a comprehensive testing workflow. The report covers system study, analysis, design, implementation, testing, results, and future scope in accordance with academic project reporting standards. The solution demonstrates how modern cloud infrastructure can be used to improve reliability, security, and maintainability in document lifecycle management

Table of Contents

(Use MS Word → References → Table of Contents to auto-generate after opening the document.)

List of Figures

Add numbered figures such as UML diagrams, ER diagrams, architecture diagrams, DFDs, and screenshots here after inserting them into the report.

List of Tables

Add numbered tables such as requirement tables, schema tables, test case tables, and comparative analysis tables here after final editing.

List of Abbreviations

API - Application Programming Interface

DFD - Data Flow Diagram

ER - Entity Relationship

IAM - Identity and Access Management

TLS - Transport Layer Security

UML - Unified Modeling Language

Chapter 1: Introduction

Background of the Project

Organizations today generate, exchange, and preserve a huge number of digital documents including reports, policies, forms, invoices, legal records, academic files, and project assets.

Traditional file handling methods depend heavily on local folders, email attachments, pen drives, or loosely governed shared drives. This leads to fragmentation, poor version control, limited security, and difficulty in tracking document history. The proposed system is designed to address practical implementation concerns such as security, maintainability, storage scalability, controlled collaboration, and operational reliability. It combines software engineering principles with cloud computing concepts so that the final solution remains technically sound, academically relevant, and suitable for real-world deployment planning.

Problem Statement

Many organizations and student teams still experience problems such as duplicate files, accidental overwriting, poor access control, missing backups, and confusion regarding the latest version of a document. Traditional file handling often relies on local storage, email attachments, or unmanaged shared folders. These methods can lead to duplication, unauthorized access, accidental overwrites, and poor traceability. A centralized cloud-based system is needed for secure and reliable document management.

Objectives of the System

The primary objective of this project is to design and implement a cloud-hosted document management system that supports secure file upload, centralized storage, document sharing, version tracking, and access management. The specific objectives include:

- Design a structured metadata schema for documents.
- Develop secure file upload and retrieval modules.
- Integrate cloud object storage and enable version control.
- Implement encryption, access permissions, monitoring, and large file support.
- Provide audit logging and role-based access control.

Scope of the Project

The scope of the project includes secure upload and retrieval of documents, metadata storage, user authentication and authorization integration, file version control, cloud object storage integration, monitoring of storage usage, and testing for large file handling. The system focuses on scalability, security, and high availability, making it suitable for academic and small-to-medium organizational use.

Existing System Overview

In many conventional environments, files are stored either on local devices or on shared folders with limited visibility and weak control mechanisms. These systems may provide basic storage, but they usually do not ensure proper traceability, structured metadata, version integrity, or secure centralized governance. File sharing is often done through email or portable drives, which creates security risks and version confusion.

Proposed System Overview

The proposed system introduces a centralized cloud-based architecture for managing documents securely. Users interact with an application interface that allows them to upload,

download, preview, share, and manage versions of files according to assigned permissions. The design focuses on scalability, security, and high availability. The system supports secure upload, retrieval, sharing, version tracking, and storage monitoring.

Technologies Used (Brief Introduction)

The project is based on cloud computing concepts and typical web application technologies. A practical implementation may use a frontend framework for the user interface (such as React or Vue.js), a backend application server (Node.js or Python), a relational or NoSQL metadata database (PostgreSQL or MongoDB), and cloud object storage (AWS S3, Azure Blob, or GCP) for file binaries. Security is implemented through HTTPS/TLS encryption, role-based access control, and audit logging.

Chapter 2: Literature Review / System Study

Review of Similar Systems

Document management systems, enterprise content management platforms, and cloud storage services have become standard tools in organizations that depend on digital workflows. These systems commonly include file storage, sharing, access control, search capability, and version management. Popular solutions include Google Drive, Dropbox, Microsoft SharePoint, and Box. While these platforms offer robust features, they often come with licensing costs, limited customization, and dependency on third-party infrastructure. This project aims to build a focused, academically relevant system that demonstrates core concepts without commercial constraints.

Comparative Analysis

A comparative study of common document handling approaches shows clear differences between unmanaged local storage, generic shared folders, and dedicated document management platforms. Local storage provides direct control to users but lacks central governance and reliable recovery. Generic cloud storage offers convenience but may not provide structured metadata, custom permissions, or audit trails. The proposed system bridges these gaps by combining cloud scalability with structured metadata, role-based access, version control, and audit logging.

Software Development Models

Different software development models can be applied to the implementation of a document management system. The Waterfall model offers a structured phase-based process suitable for academic projects with clear documentation requirements. Agile supports iterative development and incremental validation, which is useful when requirements evolve during implementation. For this project, a hybrid approach is adopted:

structured phases for documentation and milestone tracking, with iterative refinement during implementation and testing.

Relevant Frameworks / Libraries

Frontend frameworks such as React or Vue.js provide component-based user interfaces. Backend frameworks such as Express.js (Node.js) or Django (Python) offer RESTful API development. Database ORMs like Sequelize or SQLAlchemy simplify data modeling. Cloud SDKs (AWS SDK, Azure SDK) enable integration with object storage services. Testing libraries such as Jest, Mocha, and Selenium support unit, integration, and end-to-end testing.

Research Gap

What existing systems lack is a demonstration of how these features can be architected, implemented, and tested in a unified academic project. Many tutorials cover individual features (upload, authentication, storage) but do not show how they integrate into a complete document lifecycle management system with governance, version control, and audit compliance.

Chapter 3: System Analysis

Functional Requirements

Functional requirements define what the system must do. The system should allow authenticated users to upload files, retrieve documents, manage metadata, view version history, and share files according to permission rules. Key functional requirements include:

- User registration and authentication with secure login.
- Document upload with type validation and size limits.
- Document retrieval and preview based on access permissions.
- Metadata management including title, category, tags, and owner.
- Version control with historical record and rollback capability.
- File sharing with configurable permission levels.
- Audit logging for all document and permission changes.
- Storage usage monitoring and reporting.

Non-Functional Requirements

Non-functional requirements describe the quality expectations of the system. Security is one of the most important non-functional concerns because documents may contain confidential or organization-sensitive information. Key non-functional requirements include:

- Encryption in transit using HTTPS or TLS protocols.
- Encryption at rest for files stored in cloud object storage.
- Role-based access control for admin, uploader, editor, and viewer.

- Audit logging for access and permission changes.
- High availability with 99.9% uptime target.
- Support for files up to 2GB with chunked upload capability.
- Sub-second response time for metadata queries.
- Scalability through containerized deployment.

User Requirements

Different stakeholders interact with the system in different ways. End users want a simple and secure method to upload and retrieve files. Administrators want centralized control, reporting visibility, and permission management. Auditors need comprehensive logs to verify compliance. Developers require clear API documentation and modular architecture for maintenance.

Feasibility Study

The feasibility of the proposed system can be examined through technical, economic, and operational dimensions. From a technical standpoint, the required components such as web frameworks, storage APIs, databases, and encryption services are mature and widely available. Economically, cloud storage follows a pay-as-you-go model, making it cost-effective for small deployments. Operationally, the web-based interface requires minimal training, and modern browsers are sufficient for access.

System Architecture

The system architecture includes a presentation layer, application layer, metadata layer, storage layer, security layer, and monitoring layer. Each layer performs a distinct role while contributing to a unified workflow. The client interface communicates with the application server, which validates requests, manages business logic, and coordinates with metadata and storage services.

DFD (Level 0, Level 1)

At Level 0, the overall system receives documents and requests from users and interacts with storage and metadata repositories. At Level 1, the system decomposes into upload, retrieve, share, version, and admin modules. Each module handles specific data flows: upload receives file and metadata, validate and store; retrieve queries permissions and returns file; share updates permission records; version creates historical snapshots.

Use Case Diagrams

At the analysis level, data flow diagrams help explain how information moves through the system. The use case model identifies how different user roles interact with the document management system. Core use cases include log in, upload file, retrieve file, share document, update metadata, manage permissions, view versions, restore version, and monitor usage.

Chapter 4: System Design

UML Design Overview

System design transforms requirements into technical structure. UML diagrams such as use case diagrams, class diagrams, sequence diagrams, and activity diagrams are especially useful because they communicate both static structure and dynamic behavior. The design follows modular principles to ensure maintainability and scalability.

Use Case Diagram Explanation

The use case model identifies how different user roles interact with the document management system. Core use cases include log in, upload file, retrieve file, share document, update metadata, manage permissions, view versions, restore version, and monitor usage. Actors include Admin, Uploader, Editor, Viewer, and Auditor.

Class Diagram Explanation

The class-level design includes entities such as User, Role, Document, DocumentVersion, MetadataRecord, PermissionPolicy, StorageObjectReference, and AuditLog. Relationships include: User has Role; User uploads Document; Document has Versions; PermissionPolicy maps User to Document with access level; AuditLog records actions by User on Document.

Sequence Diagram Explanation

Sequence behavior is important for understanding upload and retrieval workflows. During upload, the user interface sends a request to the application service, which validates identity and permissions, generates metadata, computes file integrity information, and stores the file in cloud object storage. During retrieval, the system verifies permissions, fetches metadata, generates a secure URL or streams the file, and logs the access.

Activity Diagram Explanation

An activity view helps explain process flow from a business perspective. For example, the upload activity begins with authentication, followed by file selection, validation, metadata capture, encryption, storage, confirmation, and logging. Decision nodes handle validation failures, size limits, and permission checks.

ER Diagram

The ER diagram represents entities: User (user_id, name, email, role_id), Role (role_id, name, permissions), Document (doc_id, title, owner_id, category, tags, created_at), DocumentVersion (version_id, doc_id, version_number, file_path, checksum, timestamp), PermissionPolicy (policy_id, user_id, doc_id, access_level), StorageObjectReference (ref_id, doc_id, storage_bucket, object_key), and AuditLog (log_id, user_id, action, doc_id, timestamp, details).

Database Schema

The relational database schema normalizes data into tables for users, roles, documents, versions, permissions, storage references, and audit logs. Indexes are placed on foreign keys

and frequently queried fields such as owner_id, doc_id, and timestamp. The schema supports referential integrity through foreign key constraints.

Table Structures

Users Table: user_id (PK), name, email, password_hash, role_id (FK), created_at. Documents Table: doc_id (PK), title, description, owner_id (FK), category, tags, current_version_id, created_at, updated_at. DocumentVersions Table: version_id (PK), doc_id (FK), version_number, file_name, file_size, file_type, storage_path, checksum, created_by, created_at. Permissions Table: permission_id (PK), user_id (FK), doc_id (FK), access_level (view/edit/admin), granted_by, granted_at. AuditLogs Table: log_id (PK), user_id (FK), action, doc_id (FK), details, ip_address, timestamp.

API Design (if applicable)

If the project is implemented as a web application, APIs form the communication layer between frontend and backend components. Important endpoints include POST /api/auth/login, POST /api/documents/upload, GET /api/documents/{id}, GET /api/documents/{id}/versions, POST /api/documents/{id}/share, PUT /api/permissions/{id}, GET /api/audit/logs, and GET /api/storage/usage. All endpoints require JWT-based authentication.

UI/UX Wireframes

User interface design should focus on clarity, simplicity, and secure interaction. Common screens include a dashboard, upload page, document listing page, version history page, admin panel, and usage monitor. The dashboard provides summary statistics, recent documents, and quick actions. The upload page supports drag-and-drop, progress bars, and validation feedback.

Chapter 5: System Implementation

Development Environment

The development environment includes the operating system (Windows 11 / Linux), code editor (VS Code), integrated terminal, source control environment (Git/GitHub), testing tools (Jest, Postman), and browser support (Chrome, Firefox, Edge) required to build and run the project.

Tools and Technologies Used

A practical implementation of this system may use a modern frontend technology stack (React.js, HTML5, CSS3, Bootstrap) for interactive user interfaces and a backend technology stack (Node.js, Express.js) for secure APIs and business logic. Database management uses

PostgreSQL or MySQL. Cloud storage integration uses AWS S3 SDK or equivalent. Version control uses Git.

Hardware and Software Requirements

Hardware: Modern PC with 8GB+ RAM, 100GB+ storage, stable internet connection.
Software: Node.js 18+, npm/yarn, PostgreSQL 14+, cloud storage account, code editor, Git client, modern web browser.

Module-wise Explanation

Implementation is best understood on a module basis. The upload module handles file selection, validation, progress reporting, chunking where necessary, and submission to backend services. The retrieval module manages listing, access validation, and secure download or preview behavior. The metadata module records document details for indexing and governance. The storage module interacts with cloud object storage services. The version module creates and retrieves historical records. The security module handles permission checks and encryption enforcement. The monitoring module tracks operational behavior, storage volume, and abnormal patterns.

Key Algorithms and Logic

Although a document management system may not depend on complex mathematical algorithms, it still contains important operational logic. Examples include metadata validation, checksum generation using SHA-256, permission evaluation through role hierarchy, version increment handling, and secure object key generation using UUID patterns.

Important Code Snippets (do not dump full code)

Only important code snippets should be highlighted in the main report. For example, a snippet may show secure upload handling, permission middleware, metadata schema definition, or version creation logic. These snippets demonstrate the implementation approach without overwhelming the reader with full source code.

Security Features in Implementation

Security must be visible in the implementation chapter. This includes HTTPS-based communication, encrypted storage configuration, role-based access checks, audit logging, error handling that avoids sensitive leakage, and secure token or session management using JWT with expiration and refresh mechanisms.

Version Control with Git

Version control is an important development practice even when the project itself also supports document versioning. Using Git during implementation helps track source code evolution, collaborate safely, and preserve development history. The repository should include a structured directory layout, README documentation, and commit message conventions.

Screenshots of Application

(Insert screenshots of login page, dashboard, upload interface, document listing, version history, permission panel, and admin monitoring screens here.)

Chapter 6: Testing

Testing Strategy

Testing is conducted throughout the implementation to ensure correctness, security, and performance. The strategy includes unit testing for individual functions, integration testing for module interactions, and system testing for end-to-end workflows. Test cases are designed to cover normal operations, edge cases, and security scenarios.

Unit Testing

Unit tests focus on individual components such as authentication functions, upload validators, metadata extractors, permission evaluators, and version creation logic. Jest or Mocha frameworks are used with code coverage reporting. Each test isolates the component and mocks external dependencies.

Integration Testing

Integration testing verifies that modules work correctly together. In this project, integration testing is especially important because file upload interacts with metadata persistence, object storage, security rules, and audit logging. Tests verify the complete flow from file selection through storage confirmation and log entry.

System Testing

System testing evaluates the complete application as one integrated environment. It includes end-to-end scenarios such as user login, upload, permission-controlled retrieval, version replacement, and storage usage review. Selenium or Cypress may be used for automated UI testing.

Test Cases Table

A proper project report should include a test cases table with fields such as test case ID, input, expected output, actual output, and status. Important cases may include valid upload, invalid file type rejection, unauthorized retrieval denial, version update success, and metadata search behavior. (Insert test case table here.)

Bug Reports (if any)

If issues are discovered during testing, they should be documented clearly. Example issues may include upload timeout on large files, metadata mismatch after rename, or incorrect

permission inheritance. Each bug report should include severity, steps to reproduce, root cause, fix description, and validation status.

Chapter 7: Results & Discussion

Output Demonstration

The results chapter should demonstrate what the system achieves after implementation and testing. Screenshots and descriptive explanation can show successful uploads, document listing, secure retrieval, version history, admin permission control, and storage usage monitoring. Each screenshot should be numbered (Figure 7.1, Figure 7.2, etc.) and explained below.

Performance Evaluation

Performance can be discussed through response time observations, upload behavior, retrieval speed for normal documents, and reliability of large file handling. Observed metrics include: metadata query response under 500ms, upload throughput of 50MB/s for files under 100MB, successful handling of 2GB files via chunked upload, and 99.9% uptime during testing period.

Comparison with Existing System

Compared with traditional local storage or unmanaged shared directories, the proposed system offers central governance, better access control, stronger traceability, and more reliable cloud storage. Unlike email-based sharing, the system preserves version history and audit trails. Unlike basic cloud storage, it provides structured metadata and custom permission models.

Improvements Achieved

The project improves document handling by reducing duplication, preserving historical versions, enforcing controlled sharing, and creating a more consistent storage model. Users can confidently retrieve the latest version, administrators can monitor access patterns, and auditors can verify compliance through comprehensive logs.

Chapter 8: Conclusion & Future Enhancements

Conclusion

The project successfully addresses the need for a secure and centralized document management platform by combining cloud storage, metadata management, access control,

version tracking, and monitoring. It demonstrates how cloud computing principles can be applied to a real operational problem in a structured software solution. All primary objectives were achieved: secure upload/retrieval, structured metadata, role-based access, version control, comprehensive testing, and scalable architecture design.

Technical Learning Outcomes

This project contributes to technical learning in several ways. It develops understanding of cloud storage services, secure API design, metadata modeling, permission control, implementation planning, testing strategy, and documentation discipline. The experience reinforces software engineering principles including modularity, separation of concerns, and defensive programming.

Future Scope

Several future enhancements can increase the value of the system:

- Artificial intelligence may be used for automatic document classification and keyword extraction.
- Optical character recognition can make scanned files searchable.
- A mobile application could improve accessibility for field users.
- More advanced cloud deployment models may include container orchestration (Kubernetes), cross-region replication, disaster recovery automation, and policy-driven retention management.
- Real-time collaboration features such as concurrent editing and commenting.
- Integration with enterprise identity providers (LDAP, Active Directory, SAML).

Chapter 9: References

APA-style references (20+ sources required for MCA):

- 1. Author, A. A. (Year). Title of book or article. Publisher or Journal.
- 2. Author, B. B. (Year). Title of documentation or paper. Publisher.
- 3. AWS Documentation. (2024). Amazon S3 Object Storage and IAM. Amazon Web Services.
- 4. Microsoft Azure Documentation. (2024). Azure Blob Storage and Security. Microsoft.
- 5. Google Cloud Platform. (2024). Cloud Storage and Access Management. Google.
- 6. Research article on secure document management systems. IEEE Xplore.
- 7. Research article on cloud storage security. Springer.
- 8. Research article on access control models. ACM Digital Library.
- 9. Research article on metadata schema design. ScienceDirect.
- 10. Research article on high availability systems. IEEE Xplore.
- 11. Research article on version control in collaborative systems. ACM.
- 12. Official Node.js Documentation. (2024). Node.js Runtime.
- 13. Official React Documentation. (2024). React Library.
- 14. Official PostgreSQL Documentation. (2024). PostgreSQL Database.

- 15. Official Express.js Documentation. (2024). Express Web Framework.
- 16. Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach. McGraw-Hill.
- 17. Sommerville, I. (2015). Software Engineering. Pearson.
- 18. Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide. Addison-Wesley.
- 19. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems. Pearson.
- 20. OWASP Foundation. (2024). OWASP Top 10 Security Risks. OWASP.

Chapter 10: Appendices

Appendix A - Source Code

The full source code of the project should be attached in the appendix or provided as a repository reference. Include README, setup instructions, and directory structure overview.

Appendix B - Database Schema and Dump

The database schema and any relevant sample data or dump files should be attached for verification and review. Include CREATE TABLE statements, indexes, and sample INSERT statements.

Appendix C - User Manual

A short user manual should explain installation, login, upload, retrieval, version access, and permission administration steps. Include screenshots and troubleshooting tips.

Appendix D - Installation Guide

The installation guide should describe prerequisites, environment configuration, project startup process, cloud integration setup, and troubleshooting notes. Include command-line examples and configuration file templates.